

# Editorial: Problem B - The 727 Jumpscare

Setter: Hudson

## Problem Overview

We are given a string  $S$  consisting of digits. We need to find the number of subsequences "727". That is, the number of index triples  $(i, j, k)$  such that  $1 \leq i < j < k \leq |S|$  and  $S[i] = '7', S[j] = '2', S[k] = '7'$ .

## Naive Solution

A brute-force approach would be to iterate through all possible triples  $(i, j, k)$  and check the condition. This would require three nested loops:

```
1 long long count = 0;
2 int n = s.length();
3 for (int i = 0; i < n; i++) {
4     for (int j = i + 1; j < n; j++) {
5         for (int k = j + 1; k < n; k++) {
6             if (s[i] == '7' && s[j] == '2' && s[k] == '7') {
7                 count++;
8             }
9         }
10    }
11 }
```

The time complexity is  $O(N^3)$ . With  $|S| \leq 5000$ ,  $N^3 \approx 1.25 \times 10^{11}$  operations, which exceeds the operation limit for 1.0 second ( $\approx 10^8$  operations). This solution will result in Time Limit Exceeded (TLE).

## Optimal Solution (Dynamic Programming)

We can solve this in a single pass  $O(N)$  by keeping track of partial subsequences. The problem asks us to build "727". This is built in steps: "7"  $\rightarrow$  "72"  $\rightarrow$  "727".

Let's maintain three counters as we iterate through the string from left to right:

- **cnt7**: The number of '7's seen so far.
- **cnt72**: The number of "72" subsequences seen so far.
- **total**: The number of "727" subsequences seen so far (our answer).

### Logic:

1. When we encounter a '2': It can combine with any of the previous **cnt7** '7's to form a "72". So, we update: **cnt72** = **cnt72** + **cnt7**.

2. When we encounter a '7': This digit plays two roles.

- First, it can act as the *last* digit ( $k$ ) in a "727" subsequence. It completes any existing "72" subsequences found previously. So, we add the current number of "72"s to our answer: `total = total + cnt72`.
- Second, it can act as the *first* digit ( $i$ ) in a future "727" subsequence. So, we increment the count of single '7's: `cnt7 = cnt7 + 1`.

**Implementation Note:** When encountering a '7', the order of updates is crucial. We must update `total` before incrementing `cnt7`. If we incremented `cnt7` first, we would imply that a '7' could form a "72" with a future '2' and then complete it itself, which violates the strict index ordering  $i < j < k$ .

## Complexity Analysis

- **Time Complexity:**  $O(N)$ , where  $N$  is the length of the string. We iterate through the string exactly once.
- **Space Complexity:**  $O(1)$ , as we only use three integer variables regardless of input size.

## Implementation (C++)

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 void solve() {
7     string s;
8     cin >> s;
9
10    long long cnt7 = 0;
11    long long cnt72 = 0;
12    long long total = 0;
13
14    for (char c : s) {
15        if (c == '7') {
16            // 1. This '7' acts as the 3rd digit, completing "72..."
17            total += cnt72;
18
19            // 2. This '7' acts as the 1st digit for future sequences
20            cnt7++;
21        } else if (c == '2') {
22            // This '2' acts as the 2nd digit, extending "7..."
23            cnt72 += cnt7;
24        }
25    }
26
27    cout << total << endl;
28 }
29
30 int main() {
31     // Optimization for faster I/O
```

```
32 ios_base::sync_with_stdio(false);
33 cin.tie(NULL);
34
35 int N;
36 if (cin >> N) {
37     while (N--) {
38         solve();
39     }
40 }
41 return 0;
42 }
```